

Análisis de Algoritmos I

Complejidad y Notación asintótica

L.C.C. María Janneth Rivera Reyna

Lic. En Ciencias de la Computación
Departamento de Matemáticas
Universidad de Sonora

Ciclo 2026-1

Complejidad algorítmica

- La complejidad algorítmica representa la cantidad de **recursos** (*tiempo de ejecución**, *espacio*, etc) que necesita un algoritmo para resolver un problema y por tanto permite determinar la **eficiencia** de dicho algoritmo.
- Un algoritmo puede ser **rápido** para datos pequeños pero puede volverse **inútil** cuando el problema crece.
- Necesitamos una forma de **predecir su comportamiento**.

Complejidad algorítmica (cont.)

- Para que esta medida sea **independiente** del hardware, lenguaje de programación, compilador, habilidades de programación, etc. podemos obtenerla mediante el **número de operaciones** que ejecuta un algoritmo.
- Más formalmente, la complejidad describe **cómo crece el costo** de un algoritmo.
- Se expresa **en función del tamaño de la entrada (n)** donde n puede ser:
 - Número de datos
 - Tamaño de un arreglo
 - Número de nodos,
 - etc

Modelo RAM y las operaciones básicas

- Modelo teórico de computación llamado **Máquina RAM (Random Access Machine)**:

Cada operación i , toma el tiempo t_i

donde t_i es una constante que no depende de n

- Es decir, se asume que las operaciones básicas del CPU toman un tiempo fijo (“*cuestan una unidad*”), independientemente de si el algoritmo está procesando 10 elementos o 10 millones de elementos.

Operaciones básicas:

- Operaciones aritméticas (Ej. suma, resta, multiplicación, división)
- Comparaciones lógicas (Ej. $a < b$)
- Asignaciones a variables (Ej. $x = 1$)
- Accesos a memoria (Ej. `array[i]`)
- Llamadas o regresos de un procedimiento

Función de costo - $T(n)$

- Indica **cuántas operaciones** ejecuta un algoritmo para una entrada de tamaño n .
- Esta función **describe el comportamiento del algoritmo**.
- **Funciones de costo más comunes:**
 - $T(n) = 1$ → Crecimiento constante
 - $T(n) = n$ → Crecimiento lineal
 - $T(n) = n^2$ → Crecimiento cuadrático
 - $T(n) = \log n$ → Crecimiento logarítmico
 - $T(n) = n \log n$ → Crecimiento lineal-logarítmico
 - $T(n) = 2^n$ → Crecimiento exponencial
 - $T(n) = n!$ → Crecimiento factorial

Tasa de crecimiento y notación asintótica (O , Θ , Ω)

La *tasa de crecimiento* nos dice **cómo crece** la función de costo de un algoritmo **cuando n crece** y para ello se usa la *notación asintótica*:

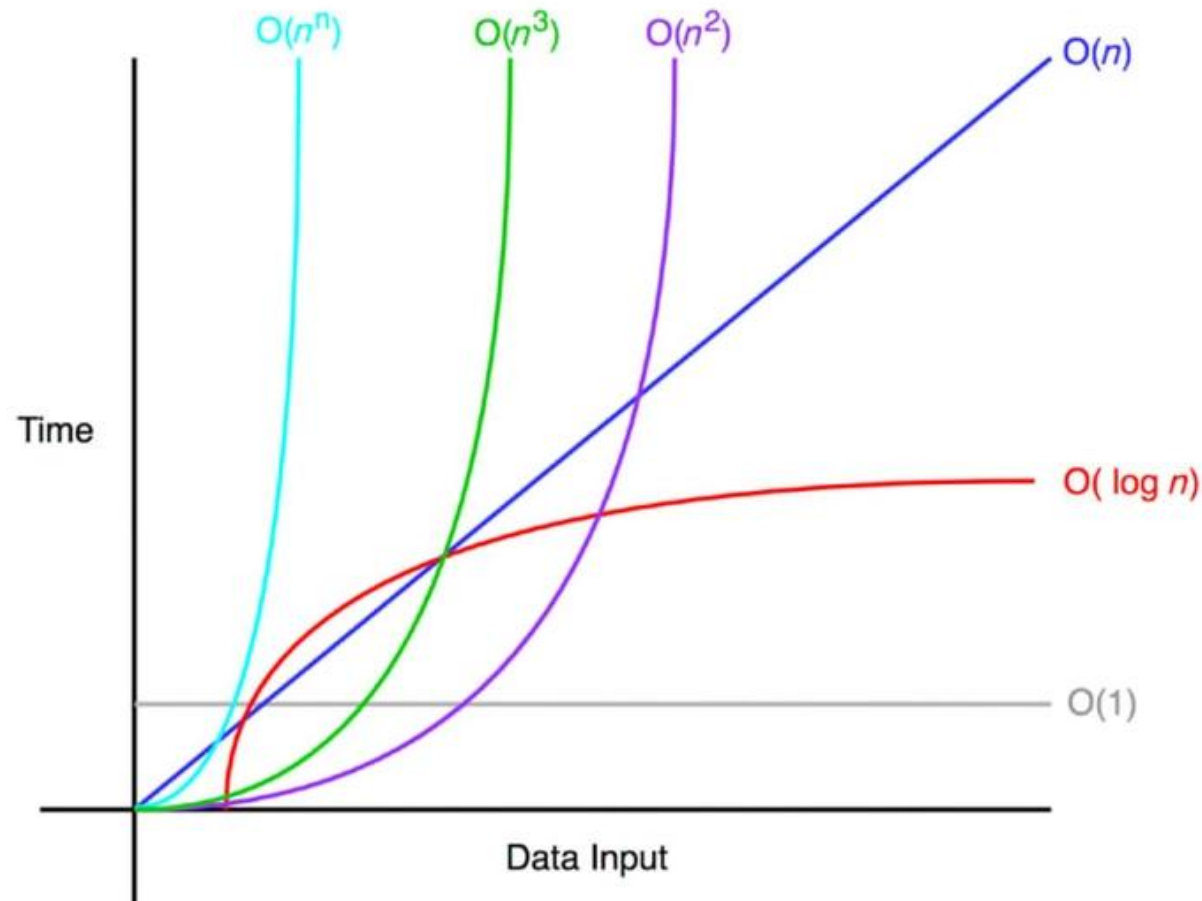
- $O()$ → cota superior (peor caso)
 - $\Theta()$ → cota ajustada (caso promedio)
 - $\Omega()$ → cota inferior (mejor caso)
- No mide el tiempo exacto
 - Describe el crecimiento del número de operaciones
 - Ignora constantes y términos pequeños
 - Ayuda a comparar algoritmos
 - Se usa principalmente para el peor caso
→ **Big-O**

De función a notación Big-O

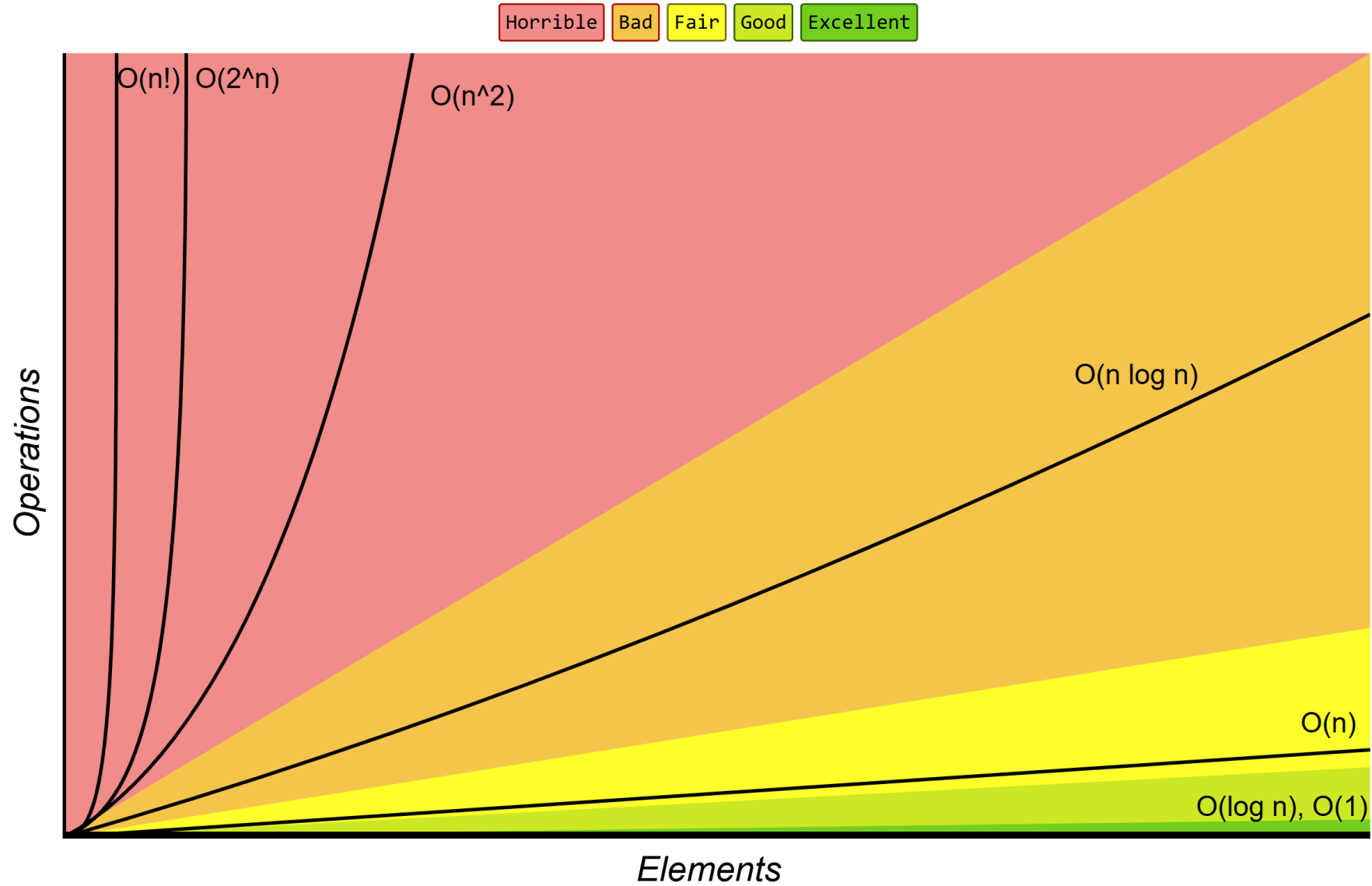
- $T(n) = 3n + 10$
 - El término dominante es n
 - Big-O: **$O(n)$**
- $T(n) = n \log n + 100n$
 - El término dominante es $n \log n$
 - Big-O: **$O(n \log n)$**
- $T(n) = 5n^2 + 3n + 10$
 - El término dominante es n^2
 - Big-O: **$O(n^2)$**

Orden de crecimiento (de mejor a peor)

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$



Big-O Complexity Chart



Análisis: Tiempo constante $O(1)$

- Su rendimiento no depende de la cantidad de datos.

```
x = 1 + 1      #1
```

Operación básica: asignación

Núm. veces que se ejecuta: 1 vez

Función de costo: $T(n) = 1$

Notación Big-O: $O(1)$

Análisis: Tiempo lineal $O(n)$

- Generalmente, los algoritmos de tiempo lineal ejecutan tareas que necesitan inspeccionar cada punto de datos individualmente.

```
for i = 1 to n:  
    x = x + 1           #n
```

Operación básica: asignación

Núm. veces que se ejecuta: n veces

Función de costo: $T(n) = n$

Notación Big-O: $O(n)$

Análisis: Tiempo cuadrático $O(n^2)$

- Estas funciones escalan mal, son adecuadas para N pequeñas, pero poco prácticas N grandes.
- Duplicar la cantidad de datos lleva a cuadruplicar el número de operaciones.

```
for i = 1 to n:
    for j = 1 to n:
        x = x + 1           #n*n
```

Operación básica: asignación (en un ciclo anidado)

Número de veces que se ejecuta: $(n * n)$ veces

Función de costo: $T(n) = (n * n) = n^2$

Notación Big-O: $O(n^2)$

Análisis: Tiempo lineal $O(n)$, otro caso

```
for i = 1 to n:  
    x = x + 1    #n  
  
for j = 1 to n:  
    y = y + 1    #n
```

Operación básica: asignación (en dos ciclos separados)

Número de veces que se ejecuta: Cada ciclo se ejecuta n veces

Función de costo: $T(n) = n + n = 2n$

Notación Big-O: $O(n)$

Análisis: Tiempo cuadrático $O(n^2)$, otro caso

```
for i = 1 to n:  
    x = x + 1          #n  
  
for j = 1 to n:  
    for k = 1 to n:  
        y = y + 1      #n*n
```

Operación básica: asignación (en dos ciclos separados)

Número de veces que se ejecuta: $n + (n*n)$ veces

Función de costo: $T(n) = n + n^2$

Notación Big-O: $O(n^2)$

Análisis: Tiempo logarítmico $O(\log n)$

- Divide repetidamente a la mitad el tamaño del problema en cada paso.
- Su tasa de crecimiento es extremadamente lenta, es muy buscada en la práctica, comportándose casi como $O(1)$.
- Para valores grandes de N , el número total de operaciones sigue siendo notablemente bajo.

```
while n > 1:  
    n = n / 2      # n/2
```

Operación básica: asignación (en un loop)

Número de veces que se ejecuta: $(n/2)$ veces

Función de costo: $T(n) = \log n$

Notación Big-O: $O(\log n)$